

Memory and Context Engineering

CMSC 848N

Fall 2026, Week 5, Lecture 1

Based on the EMNLP 2024 tutorial by Yu Su, Diyi Yang, Shunyu Yao, and Tao Yu

The agent harness

The harness is the software that runs an agent across model calls.

Example task

“Compare three restaurants for Thursday dinner.”

Stored preference: vegan food

Current instruction: find options only. Do not book.

How will each model call receive the information it needs?

The execution loop

```
for step in range(step_limit):  
    context = build_context(task, history, memory)  
    reply = model(context, allowed_tools)  
    if reply.is_final:  
        return reply.answer  
    calls = validate_and_authorize(reply.tool_calls)  
    observations = execute_with_limits(calls)  
    history.append(reply, observations)  
return incomplete(history)
```

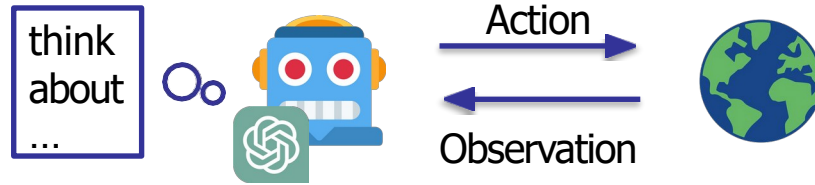
Today: which records should build_context include?

What the next model call needs

Model output	Harness action and next context
Search for suitable restaurants	Search tool returns candidate restaurants.
Read menus and prices	Tool results enter history with source links.
Compare the options	The answer uses menu evidence and the vegan preference.

The harness permits searches and reads. Booking is disallowed.

What should an agent remember?



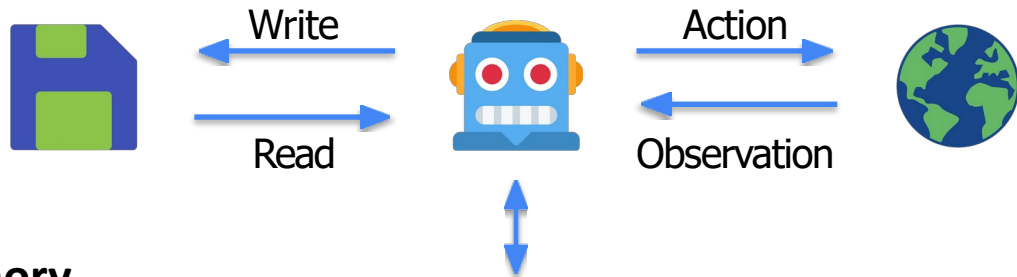
A task continues after the visible page changes.

The agent has seen a room, tried an action, and received feedback.

Which observations will it need after leaving the room?

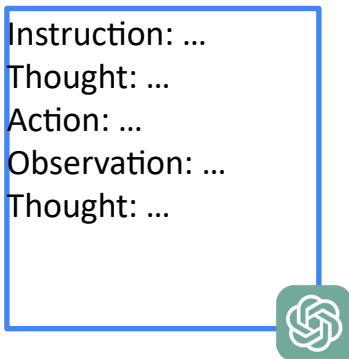
Which experiences will still matter on a later task?

Working context and persistent memory



Persistent memory

Stores records outside the current model call.
The system can read, revise, or delete them.



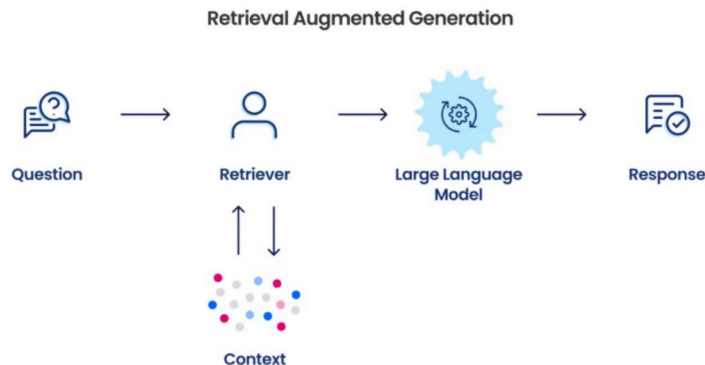
Working context

Contains the tokens supplied now.
The harness selects history, retrieved records, and tool results.

Persistence across calls depends on what the system stores and reloads.

Retrieval and agent memory

- RAG retrieves records and supplies them to a language model.
- A fixed external corpus is one configuration. An agent can also retrieve its own stored experiences.
- Read and write policies determine what persists and how the corpus changes.



The retrieval corpus can contain documents, event records, or both.

Memory types describe the content

Type by content	Definition	Examples
Episodic memory	Stores experience	Generative agents [Park et al., 2023]
Semantic memory	Stores knowledge	
Procedural memory	Stores skills	Voyager [Wang et al., 2023]

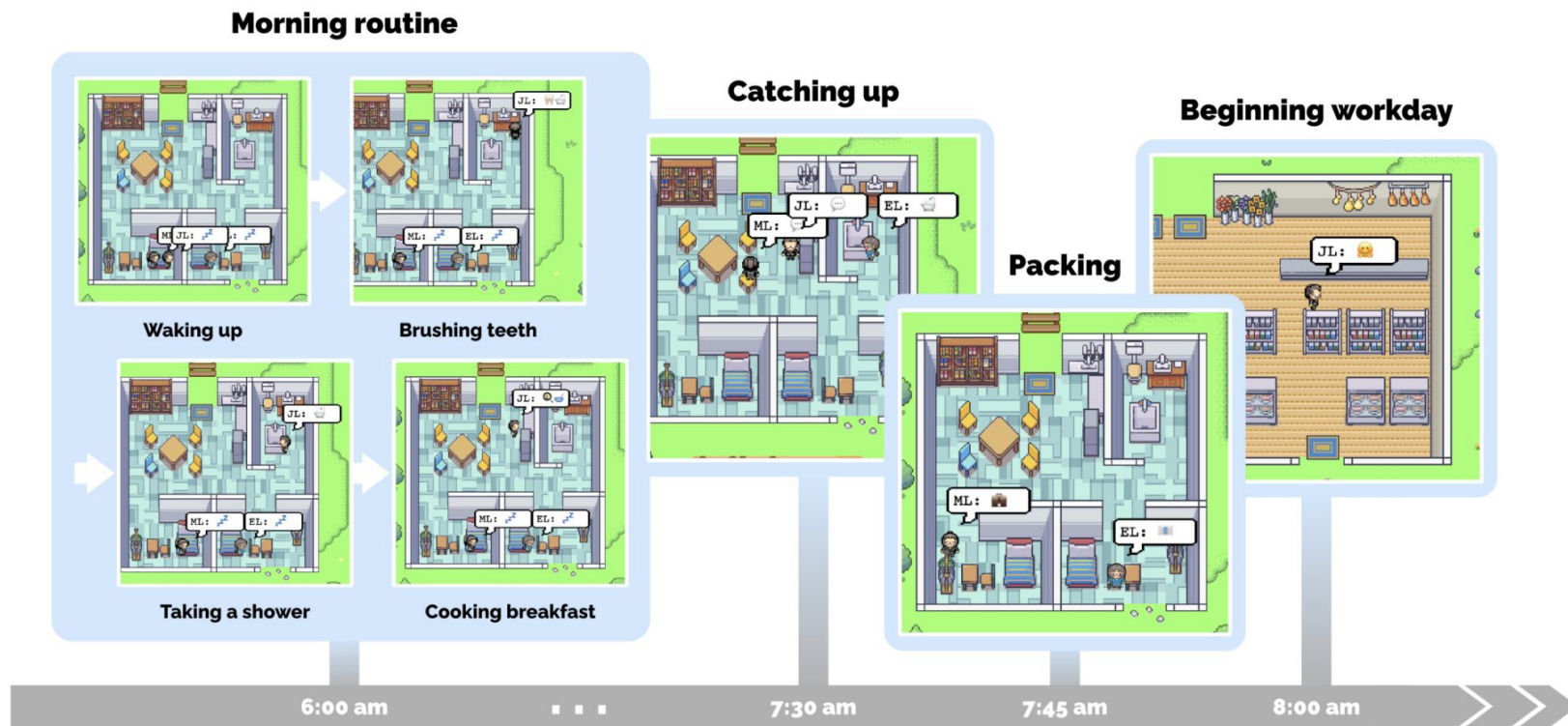
The categories describe what a memory contains.

Text files, vector indexes, graphs, and model weights are implementation choices.

Generative Agents in Smallville



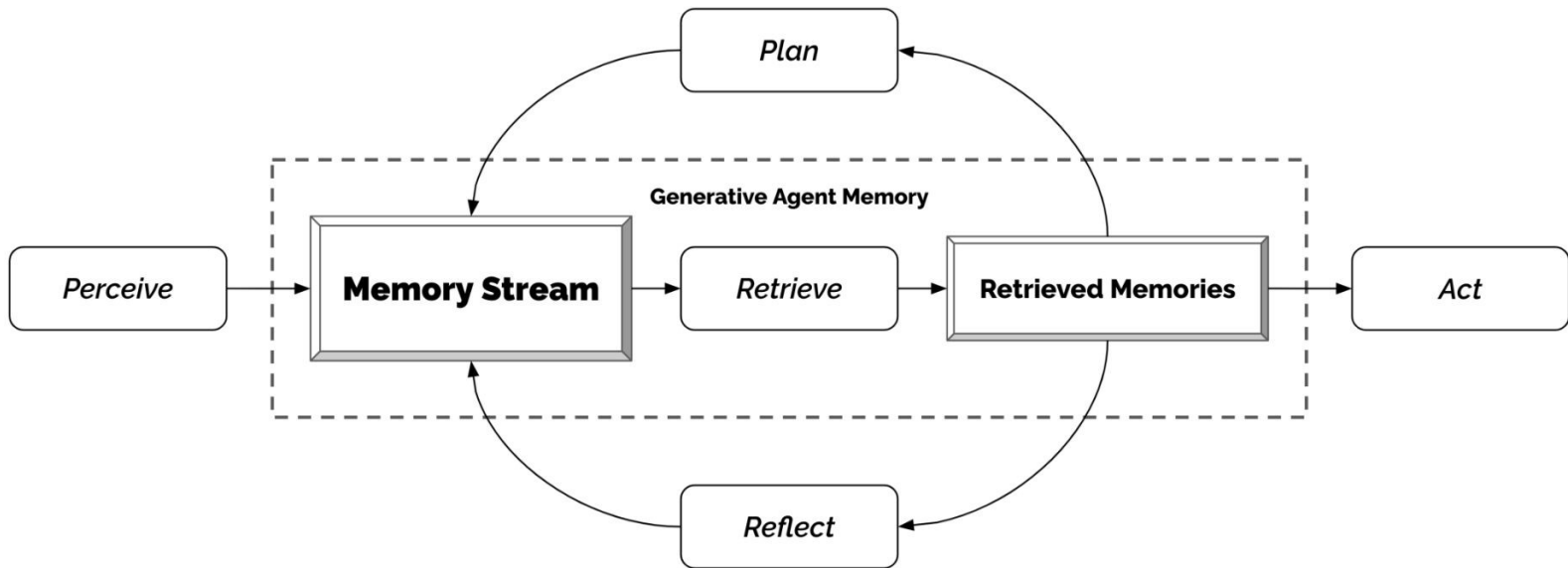
An agent follows a daily routine



The memory architecture in Generative Agents

Event histories can outgrow the context window.

Retrieval selects records. Reflection derives higher-level memories.



Episodic memory stores events

Writing appends observations. Reading ranks stored events for the current situation.

Memory Stream

2023-02-13 22:48:20: desk is idle
2023-02-13 22:48:20: bed is idle
2023-02-13 22:48:10: closet is idle
2023-02-13 22:48:10: refrigerator is idle
2023-02-13 22:48:10: Isabella Rodriguez is stretching
2023-02-13 22:33:30: shelf is idle
2023-02-13 22:33:30: desk is neat and organized
2023-02-13 22:33:10: Isabella Rodriguez is writing in her journal
2023-02-13 22:18:10: desk is idle
2023-02-13 22:18:10: Isabella Rodriguez is taking a break
2023-02-13 21:49:00: bed is idle
2023-02-13 21:48:50: Isabella Rodriguez is cleaning up the kitchen
2023-02-13 21:48:50: refrigerator is idle
2023-02-13 21:48:50: bed is being used
2023-02-13 21:48:10: shelf is idle
2023-02-13 21:48:10: Isabella Rodriguez is watching a movie
2023-02-13 21:19:10: shelf is organized and tidy
2023-02-13 21:18:10: desk is idle
2023-02-13 21:18:10: Isabella Rodriguez is reading a book
2023-02-13 21:03:40: bed is idle
2023-02-13 21:03:30: refrigerator is idle
2023-02-13 21:03:30: desk is in use with a laptop and some papers on it

...

Q. What are you looking forward to the most right now?

Isabella Rodriguez is excited to be planning a Valentine's Day party at Hobbs Cafe on February 14th from 5pm and is eager to invite everyone to attend the party.

retrieval		recency		importance		relevance
2.34	=	0.91	+	0.63	+	0.80

ordering decorations for the party

2.21	=	0.87	+	0.63	+	0.71
------	---	------	---	------	---	------

researching ideas for the party

2.20	=	0.85	+	0.73	+	0.62
------	---	------	---	------	---	------

...

I'm looking forward to the Valentine's Day party that I'm planning at Hobbs Cafe!



Memory ranking in Generative Agents

$$\text{score}(m, q) = \alpha_r R(m) + \alpha_i I(m) + \alpha_s S(m, q)$$

$R(m)$: recency, with exponential decay since the last access.

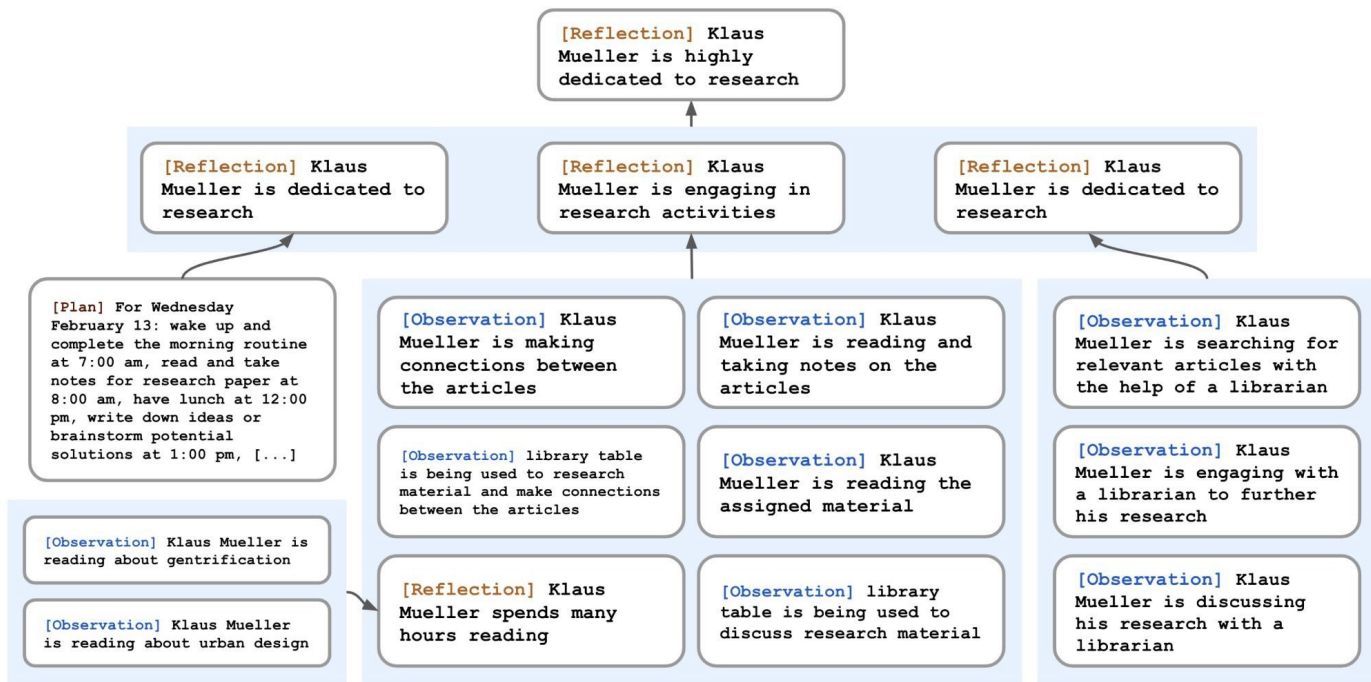
$I(m)$: importance assigned to the event by the language model.

$S(m, q)$: cosine similarity between the memory and query embeddings.

The paper normalizes each component to $[0, 1]$ and uses equal weights.
The highest-ranked records that fit the context budget enter the prompt.

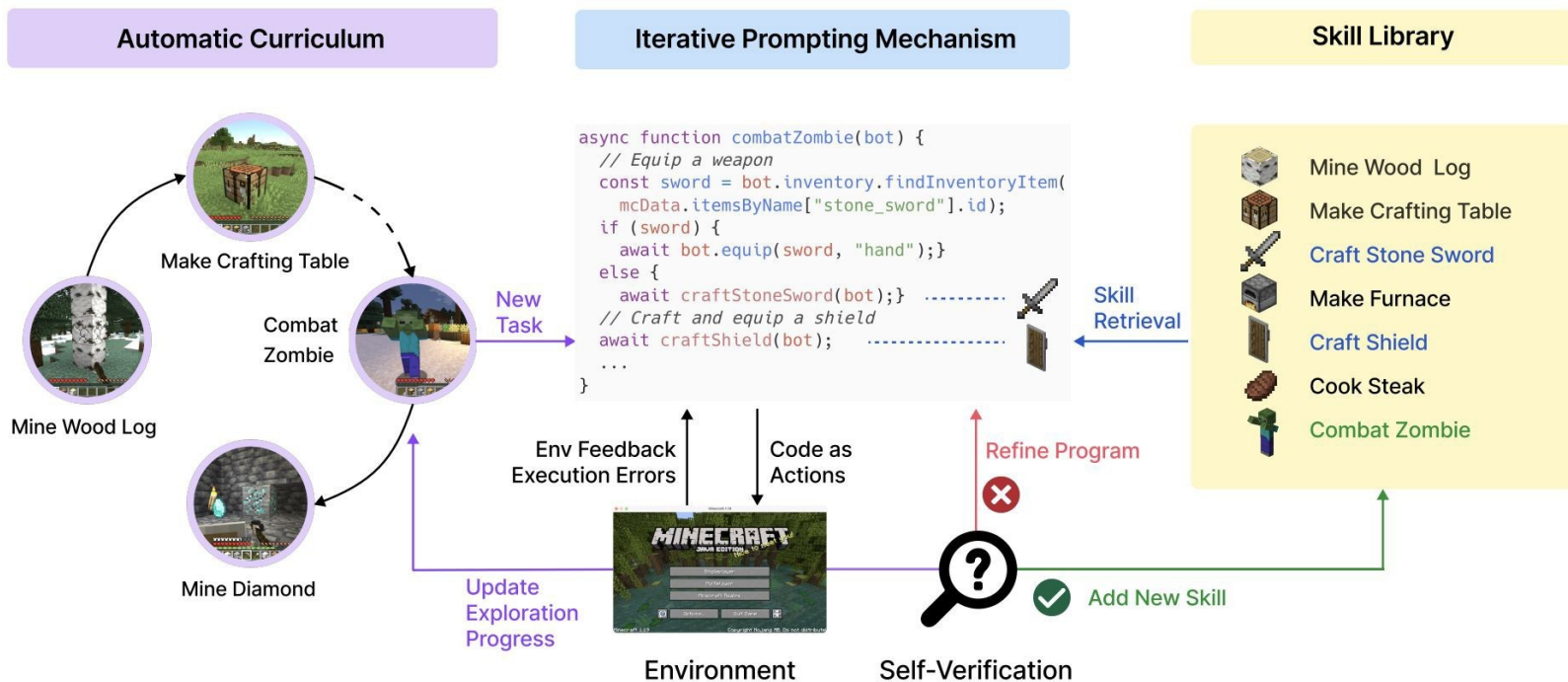
Reflection derives semantic memories

The model combines events into an inference, then stores it for later retrieval.



Voyager stores executable skills

Successful programs enter a skill library. Description embeddings support retrieval.



Memory indexes support different read operations

Text index

Store records as text. Retrieve with BM25 or embedding similarity.

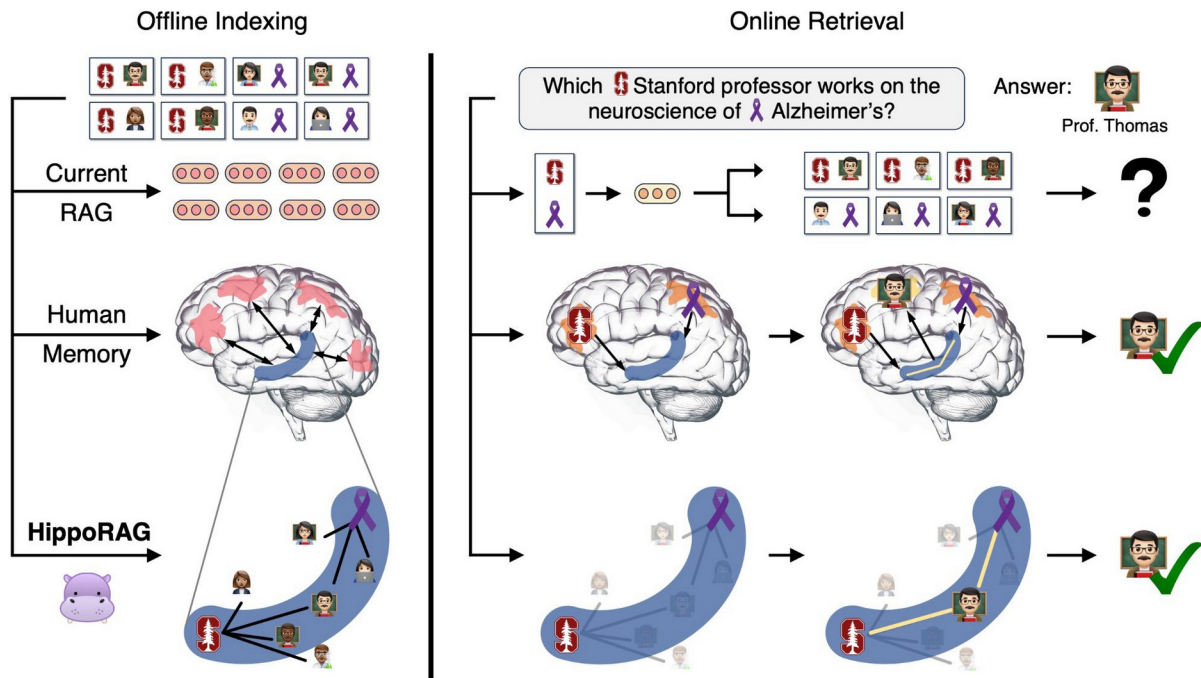
Graph index

Store entities and relationships. Retrieval can follow connections across records.

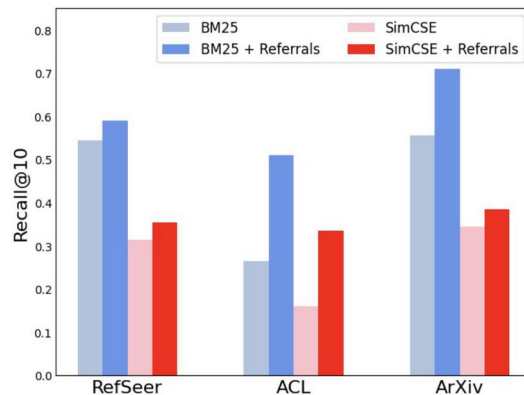
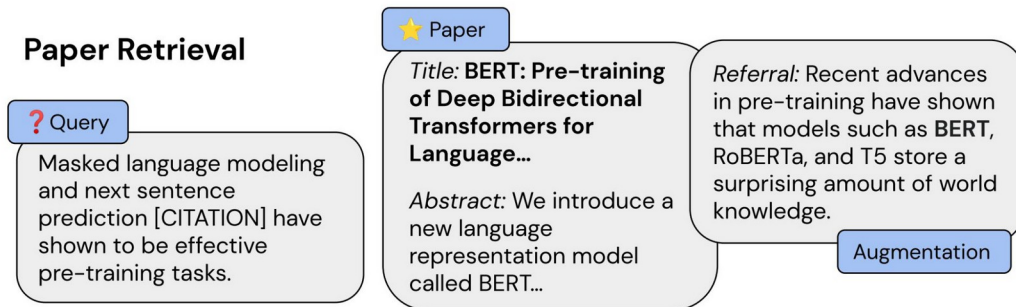
Index construction and query-time retrieval incur different costs.

HippoRAG retrieves connected facts

Query entities seed Personalized PageRank over a knowledge graph.



Referral augmentation enriches the index

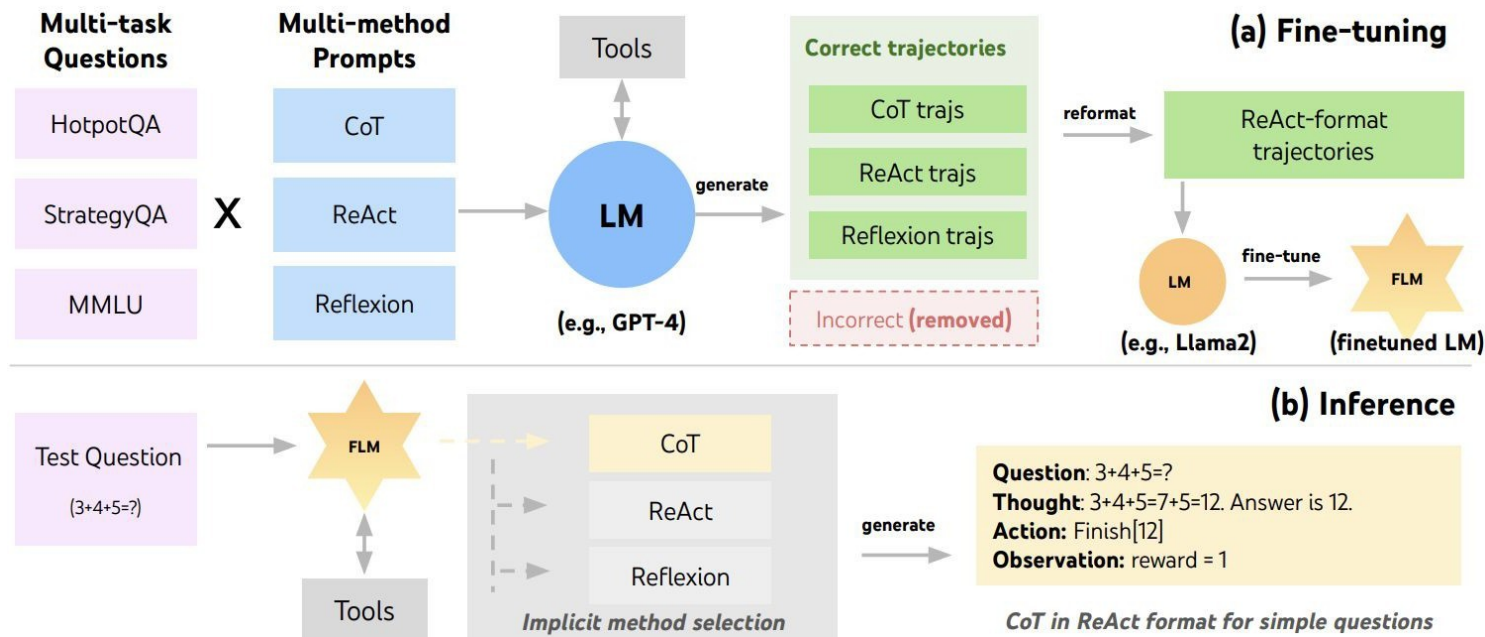


A referral is text in another document that cites or links to the target.
Adding referrals gives the retriever additional descriptions of that target.

The chart reports Recall@10 for the paper's retrieval setting.

Learning through model weights

FireAct fine-tunes a model on successful reasoning-and-action trajectories.



Learning through prompts

APE proposes candidate instructions, scores them on examples, and selects a prompt.

✓ Keep the high score candidates ✗ Discard the low score candidates ★ Final selected prompt with highest score

LLMs as Inference Models

Professor Smith was given the following instructions: <INSERT>

Here are the Professor's responses:

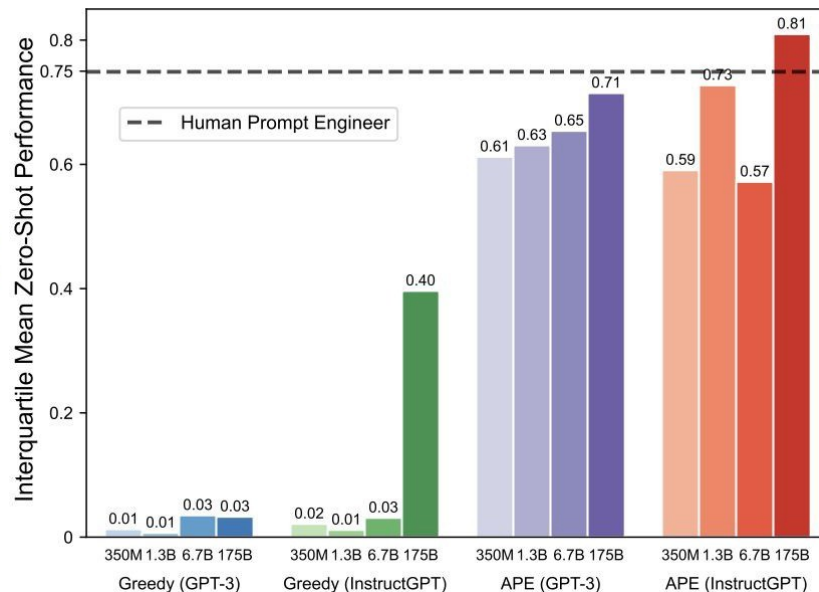
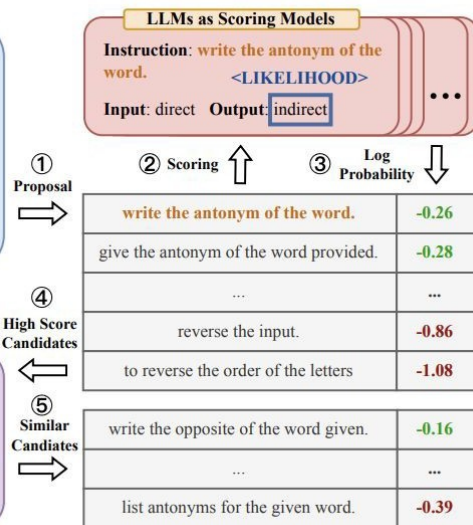
Demonstration Start
Input: prove Output: disprove
Input: on Output: off
...
Demonstration End

[Optional]

LLMs as Resampling Models

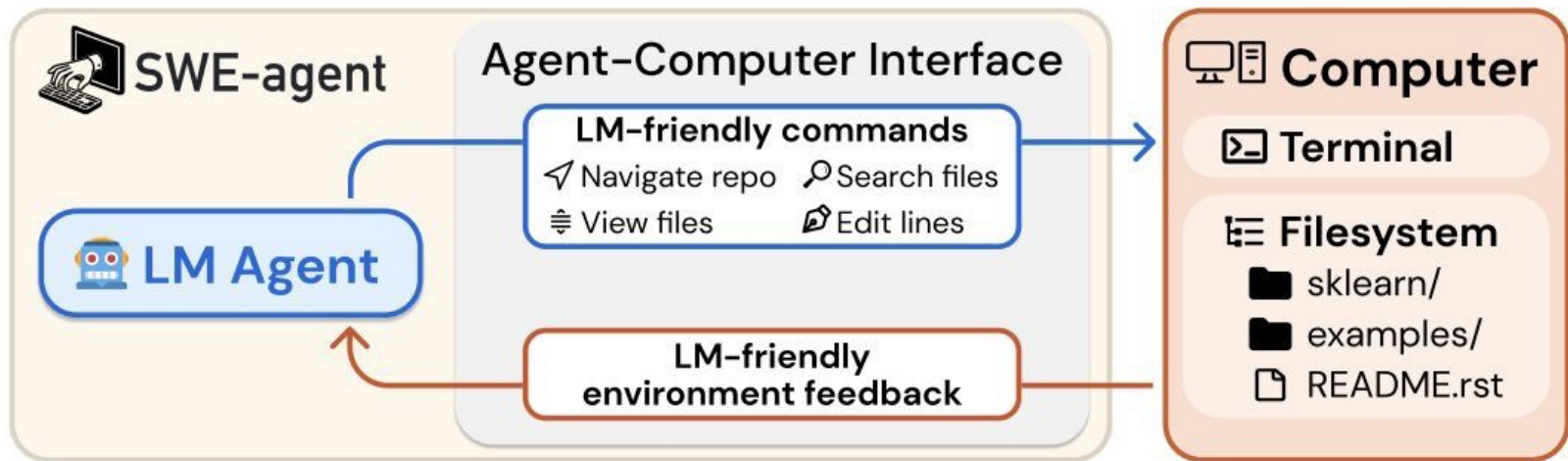
Generate a variation of the following instruction while keeping the semantic meaning.

Input: write the antonym of the word.
Output: <COMPLETE>



Code changes can persist as task artifacts

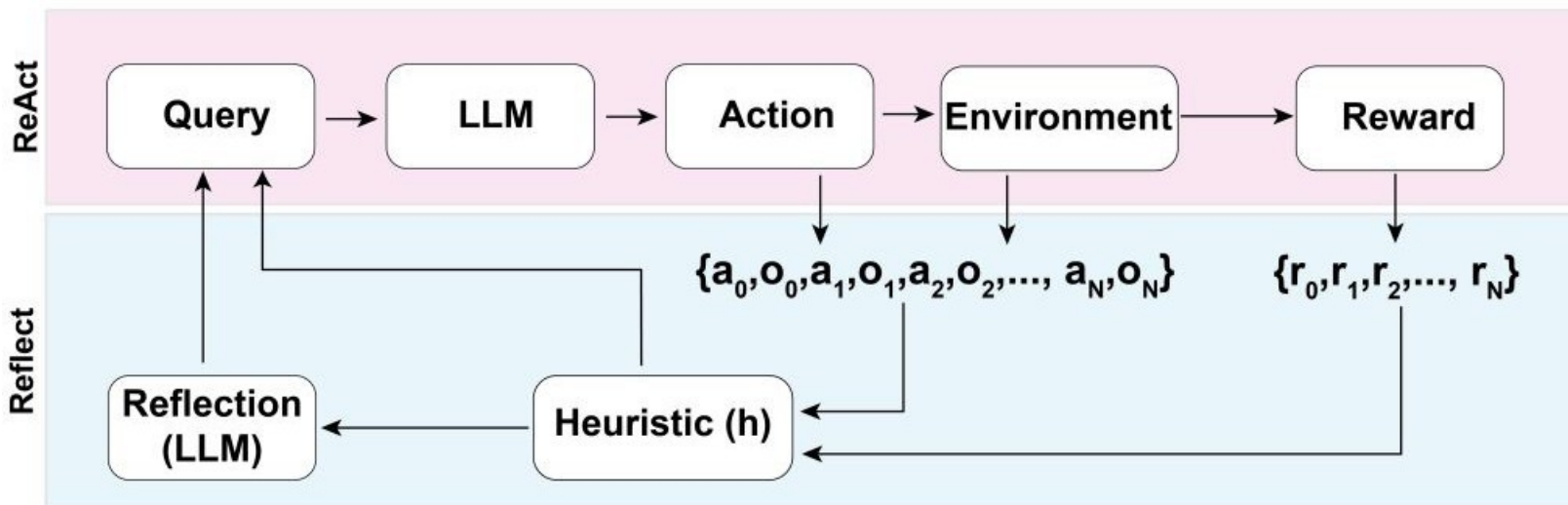
SWE-agent edits a target repository through an agent-computer interface. The resulting patch can persist after the interaction ends.



Reflexion carries feedback into the next attempt

A failed attempt produces feedback and a textual reflection.

A later attempt can retrieve that reflection while the model weights stay fixed.



Combining prompt and weight optimization

BetterTogether alternates prompt optimization and fine-tuning within an LM pipeline.

Fine-Tuning and Prompt Optimization: Two Great Steps that Work Better Together

Dilara Soylu Christopher Potts Omar Khattab

Stanford University

Algorithm 1 BetterTogether: Optimizing LM programs by alternating prompt and weight optimization steps, instantiated in **Algorithm 2**

Input: Program $\Phi_{\langle\Theta,\Pi\rangle} = \Phi_{\Theta} \odot \Phi_{\Pi}$,
with module weights $\Theta = [\theta_1, \dots, \theta_{|\Phi|}]$
and module prompts $\Pi = [\pi_1, \dots, \pi_{|\Phi|}]$

Training Set X and **Metric** μ

```
1: function BETTERTOGETHER( $\Phi_{\langle\Theta,\Pi\rangle}, X, \mu$ )
2:    $\Pi' \leftarrow \text{OPTIMIZEPROMPTS}(\Phi_{\langle\Theta,\Pi\rangle}, X, \mu)$ 
3:    $\Theta' \leftarrow \text{FINETUNEWEIGHTS}(\Phi_{\langle\Theta,\Pi'\rangle}, X, \mu)$ 
4:    $\Pi'' \leftarrow \text{OPTIMIZEPROMPTS}(\Phi_{\langle\Theta',\Pi\rangle}, X, \mu)$ 
5:   return  $\Phi_{\langle\Theta',\Pi''\rangle}$ 
6: end function
```

Selecting context under a token budget

Record i has token length ℓ_i and estimated utility u_i .

Select it with $z_i = 1$, or omit it with $z_i = 0$.

$$\begin{array}{ll}\text{maximize} & u_1 z_1 + \cdots + u_n z_n \\ \text{subject to} & \ell_1 z_1 + \cdots + \ell_n z_n \leq B\end{array}$$

B is the remaining budget after fixed instructions, the current task, and an output allowance.

Additive utility ignores duplication and dependencies between records.

Example: an updated preference and a limit

Current request: compare three restaurants for Thursday dinner.

Date	Stored record	What matters now
Sep 17	"I eat vegetarian food."	Earlier preference
Sep 22	"I now eat vegan food."	Updated preference
Sep 22	"Find options only. Do not book."	Action limit
Sep 24	Restaurant menus and prices	Current evidence

Full history, retrieval, and summaries

Strategy	What it preserves	Failure to test
Full history	Original records and wording	Old and new preferences compete for attention.
Retrieval	A small set of relevant records	A query may miss the update or action limit.
Rolling summary	A compact account of the task	Compression may omit a condition or source.

A hybrid can keep recent records, retrieve older evidence, and retain a short task summary.

Compacting a task history

A useful summary of the example

Compare three restaurants for Thursday dinner.

Use the Sep 22 vegan preference.

Do not make a reservation.

Verify current menus and prices before recommending options.

Keep links to the original records. A later question may require details the summary omitted.

Memory updates need scope and provenance

A record needs a source, a timestamp, and a scope of use.

An explicit update can supersede an earlier preference. Conflicting reports may require verification.

A retrieved document supplies evidence. It does not grant permission to act.

Deletion and access controls must cover indexes, summaries, and cached copies.

In the example, what evidence would permit making a reservation?

Evaluating memory on the same tasks

Hold the model, tools, tasks, and output budget fixed.

Compare no memory, full history, retrieval, and summary-based context.

Task outcome: does the agent answer correctly or finish the task?

Updates: does it use the latest valid record and recognize missing evidence?

Constraints: does it preserve action limits and respect access controls?

Cost: how many input tokens, retrieval calls, and seconds does it use?

Memory and the next decision

Persistent changes: stored records, reusable code, prompts, and model weights.

Read and write operations: retrieval, reflection, updates, and deletion.

Working context: the evidence and instructions available for this decision.

Where should we draw the boundary between memory and the environment?

